

Kushal Kumar Reddy K

Denton, TX-76205 | 469-343-4173 | kushall0100@gmail.com

<https://www.linkedin.com/in/kushallk/>

PROFESSIONAL SUMMARY

Full-stack software engineer with 4+ years of experience building customer-facing web applications and internal enterprise platforms using **React, TypeScript, Node.js, Spring Boot, Kafka**, and **AWS**. Delivered production features in a high-traffic e-commerce environment at Nykaa - a publicly listed Indian fashion and beauty platform with **10M+ monthly users** - and currently own the full delivery cycle for a multi-tenant SaaS product at EdgeWave. Integrates AI-assisted development tools into daily engineering workflow to accelerate delivery without compromising code quality or test coverage. Also built AI-driven and data-intensive applications through graduate research and personal projects. Targeting frontend and full-stack engineering roles in the U.S.

TECHNICAL SKILLS

Frontend: React.js, Next.js, TypeScript, JavaScript (ES6+), Redux Toolkit, Zustand, React Query, Tailwind CSS, HTML5, CSS3, Styled Components, Webpack, Vite.

AI & Applied ML: OpenAI API, LLM integration, prompt engineering, RAG fundamentals, FastAPI, scikit-learn, SHAP/LIME, Python (pandas, NumPy)

AI Dev Tools: GitHub Copilot, Cursor, ChatGPT (code review & documentation), AI-assisted test generation

Backend: Node.js, Express.js, Java 11+, Spring Boot, Spring MVC, GraphQL, REST APIs, Microservices, Apache Kafka, JWT, OAuth 2.0

Cloud & DevOps: AWS (EC2, S3, Lambda, IAM, CloudWatch, Route 53), Docker, GitHub Actions, Jenkins, Azure DevOps, Terraform (basic)

Databases: PostgreSQL, MongoDB, MySQL, Redis, SQL Server

Testing: Jest, React Testing Library, Cypress, Mocha, Chai, Postman, Swagger, TDD/BDD

Tools & Methods: Git, GitHub, JIRA, Confluence, Agile/Scrum, System Design, Figma, D3.js, WebSockets

PROFESSIONAL EXPERIENCE

Software Engineer - EdgeWave Technologies. (Remote)

September 2023 – Present

B2B SaaS company building enterprise workflow automation software for HR, compliance, and operations teams.

- Designed multi-tenant GraphQL and REST APIs in Node.js/Express with JWT-based role-based access control, supporting full data isolation across 15+ enterprise accounts. Rewrote the access-control middleware after discovering permission-scoping gaps that only surfaced under concurrent requests during load testing.
- Cut frontend initial load time by 35% by auditing the Webpack bundle, removing three unused vendor libraries (~180KB combined), and introducing route-level code splitting. The improvement was measurable on lower-bandwidth enterprise connections where the previous load time was creating drop-off before the app became interactive.
- Integrated GitHub Copilot into the team's development workflow for unit test generation and boilerplate scaffolding across Node.js services - reduced time spent on routine test writing by roughly 40%, which freed up review cycles for architecture decisions and edge-case coverage that tools don't catch on their own.
- Built GitHub Actions pipelines covering lint, unit tests, staging preview deployments, and rollback. Deployment failure rate dropped over 60% within the first quarter - most failures were caused by environmentally variable mismatches that CI now catches before anything reaches production.
- Maintained 85%+ unit and integration test coverage using Jest and React Testing Library, enforced through pre-commit hooks. Caught two separate auth-flow regressions before they reached staging, both of which would have broken login for a subset of users on session expiry.
- Integrated AWS EC2, S3, IAM, and CloudWatch for backend deployments, file handling, and infrastructure monitoring. Set up metric alarms that flag a memory leak in a Node worker process within 20 minutes of it appearing in production - before any user reported an issue.

Software Engineer- Nykaa, Hyderabad, India.

February 2020 – November 2022

Project: High-Traffic E-Commerce Platform for Fashion & Beauty

- Owned the React.js and TypeScript codebase for product listing, cart, and checkout - three of the highest traffic flows on the platform. Implemented WCAG 2.1 accessibility improvements and responsive layout fixes across those pages; mobile conversion showed a measured lift in subsequent A/B testing.
- Built four Spring Boot microservices covering pricing rules, inventory counts, promotional discounts, and order state. Extracting these from the monolith let each service be deployed and scaled independently, which became important during sale events where inventory and pricing traffic patterns differed significantly.
- Integrated Apache Kafka as the messaging layer between inventory, order, and notification services. During the Nykaa Beauty Fest - where traffic consistently hits 6–8x normal volume - the Kafka-backed pipeline held without the queuing delays that had caused missed order updates in the earlier Redis-based setup.
- Replaced REST endpoints on product listing pages with GraphQL, letting the frontend request only the fields each view needed. Average API payload dropped around 40%, and time-to-interactive on product pages improved noticeably on mid-range Android devices, which make up a large share of Nykaa's user base.
- Implemented JWT authentication with refresh token rotation across the web app and the PWA. Handled the edge case where tokens expired mid-checkout, which had been silently logging out a small percentage of users at the payment step - one of the harder-to-reproduce bugs on the platform.
- Containerized six Java microservices with Docker, deployed to AWS EC2 behind an ALB, and configured CloudWatch alarms for CPU, memory, and 5xx error rates. Led UAT within Agile sprints - incident escalations from production dropped through the final four quarters, and feature acceptance rates averaged around 95% across that period.

PROJECTS

StreamIQ - Real-Time Analytics Pipeline · Kafka · Spring Boot · FastAPI · React · WebSockets. github.com/K-15a-FullstackDev/streamiq

- Built an event pipeline where raw events move through separate ingestion, enrichment, inference, and presentation stages: a Spring Boot service enriches events, a Python FastAPI service runs anomaly checks, and a React dashboard renders results in real time via WebSockets. Each stage is a separate deployable unit.
- Kept the Python inference service isolated from the Java backend even though it made local setup harder. The tradeoff was worth it - the ML side can be iterated on, swapped, or scaled without touching the enrichment layer.

Explainable AI Dashboard - ML Interpretability Platform · React · Python · FastAPI · SHAP/LIME · scikit-learn · AWS EC2. github.com/K-15a-FullstackDev/explainable-ai-dashboard

- Full-stack platform where users upload tabular datasets, trigger model training, and explore SHAP and LIME explanations through a React interface backed by a Python FastAPI service. Built during the MS Data Science program to close the gap between a model's output and something a non-technical stakeholder could interpret.
- Supports both global feature importance and local per-row explanations - two different questions that many similar tools conflate. Keeping them as separate workflows was a deliberate design decision, not an afterthought.

Teamboard - Project Management Tool · React · Node.js · PostgreSQL · JWT · AWS S3 · GitHub Actions · Docker. github.com/K-15a-FullstackDev/Teamboard

- Kanban-style task management app with sprint tracking, role-based access for admins and members, and file attachments via S3. GitHub Actions pipeline runs tests, builds a Docker image, and deploys EC2 on merge to main.
- Built auth and the data model before touching any UI - most task board projects bolt auth after the fact and spend the rest of the project patching around a weak permission structure. Getting that right first meant every feature after it built on something stable.

EDUCATION

M.S., Data Science - University of North Texas, Denton, TX

Dec 2024 · GPA: 3.83 / 4.00

Relevant coursework: Machine Learning, Statistical Modeling, Big Data Systems, Data Visualization, Cloud Computing for Data Science.